

E-Commerce Data Analytics Platform

End-to-End Business Intelligence & Data Engineering Case Study

1. Executive Summary & Business Problem Statement

Overview: The international E-Commerce Bookstore faced significant challenges due to siloed data spanning sales, product inventory, and sales team performance. The primary objective of this project was to architect a comprehensive "Single Source of Truth" (SSOT) data platform. This robust system tracks revenue velocity, accurately calculates sales rep commissions, performs behavioral customer RFM segmentation, and provides real-time monitoring of inventory health.

Strategic Business Goal

To empower senior leadership with a scalable, data-driven framework that optimizes Return on Investment (ROI), enhances customer retention strategies, and significantly reduces inventory shrinkage and stockouts.

Technology Stack

Data Engineering:

Python

BeautifulSoup

Requests

Faker

Pandas

NumPy

Data Warehousing:

Microsoft SQL Server (T-SQL)

Star Schema

Analytics & BI:

Microsoft Excel (Pivot Analytics)

Power BI (DAX)

2. End-to-End System Architecture

The data pipeline was engineered across four distinct phases to guarantee data integrity, automated flow, and seamless integration from raw sources to the final BI dashboard:

1. Data Ingestion & Simulation:

- **Web Scraping:** Extracted core product data for 50 unique books from 'Books to Scrape' (capturing Product Name, Price, Star Rating, Stock Availability, and Category) using BeautifulSoup.

- **Synthetic Generation:** Leveraged Python's Faker library to simulate 2,000 realistic global customers and 40 Sales Representatives across B2B, Retail, and Online Sales departments, incorporating variable base salaries and commission rates.
- **Transaction Modeling:** Generated a Fact table comprising 12,500 validated order transactions across the 2023-2024 fiscal period, meticulously mapping relationships between Customers, Products, and Employees.

- 2. **Data Cleaning & ETL (Pandas):** Executed deep data sanitization, including currency string normalization, Regex-based stock digit extraction, and date format standardization.
- 3. **Data Warehousing (SQL Server):** Loaded the transformed datasets into a robust Star Schema optimized for high-performance analytical queries.
- 4. **Business Intelligence (Power BI):** Connected the dimensional model to Power BI using DirectQuery/Import modes to construct the semantic data model and visualize metrics.

3. Data Dictionary & Relational Schema

The enterprise Data Warehouse was designed using a standard **Star Schema** to ensure referential integrity and optimize read-heavy BI workloads.

TABLE NAME	TYPE	ROWS	DESCRIPTION & KEY ATTRIBUTES
Fact_Orders	Fact	12,500	Keys: FK CustomerID, FK ProductID, FK EmployeeID. Metrics: OrderDate, Quantity, TotalAmount, DiscountApplied.
Dim_Customers	Dimension	2,000	Keys: PK CustomerID. Attributes: Full Name, Email, Country, Tenure, Email Domain.
Dim_Products	Dimension	50	Keys: PK ProductID. Attributes: Title, Category, Price, StarRating, StockQty.
Dim_Employees	Dimension	40	Keys: PK EmployeeID. Attributes: Name, Department, BaseSalary, CommissionRate.

4. ETL & Data Cleaning Methodology

Using Python's Pandas library, raw ingested data was heavily transformed to ensure strict adherence to database constraints. Key transformations included:

- **Currency Cleansing:** Stripped non-numeric currency symbols and cast columns to float arrays for aggregate math operations.

- **Date Normalization:** Cast all temporal data into strict ISO 8601 standard datetime formats.
- **Feature Engineering:** Extracted email domains to analyze B2B vs B2C proportions and computed customer "Tenure" in days to feed into loyalty models.

Challenge Solved: Stock Digits Extraction

Raw scraped data contained inconsistent string formats such as *"In stock (19 available)"*. Regular Expressions (Regex) were utilized to isolate integers natively within the dataframe vector space:

```
# Extracting numerical stock values from messy string columns
df['Stock_Availability'] = df['Stock_Availability'].str.extract(r'(\d+)').astype(int)
df['Price_Clean'] = df['Price'].str.replace(r'^\d.', '', regex=True).astype(float)
```

5. Business Intelligence & Key Analytical Insights

The backend T-SQL layer processes 12 advanced analytical queries to feed the BI semantic model. Key insights include:

1. **Executive Performance (KPIs):** Total Revenue, Total Transaction Count, Average Order Value (AOV), and Active Customer Count.
2. **Time-Series Velocity:** Leveraged SQL Window Functions (LAG () and LEAD ()) to calculate precise Month-over-Month (MoM) revenue growth, identifying critical Q4 seasonal spikes.
3. **Customer RFM Segmentation:** Implemented Recency, Frequency, and Monetary (RFM) modeling using the NTILE (5) function, automatically clustering customers into targetable segments (e.g., 'VIP', 'Loyal', 'Churn Risk').
4. **Sales Team Analytics:** Calculated Departmental ROI (Revenue generated vs. Base Salary outlay), automated commission structures, and ranked the Top 10 Sales Representatives via dense ranking.
5. **Inventory Health:** Cross-referenced low-stock warnings with sales velocity to identify high-margin items at risk of stockouts.

6. Strategic Business Recommendations

Actionable Next Steps for Stakeholders

- **Targeted Marketing Execution:** Deploy specialized retention campaigns specifically aimed at the 'VIP' segment identified in the RFM analysis, while offering aggressive discount codes to the 'Churn Risk' segment.
- **Automated Supply Chain Triggers:** Integrate the Power BI "Low Stock Alerts" directly into the ERP/Procurement system via Power Automate to re-order high-velocity books before stockouts occur.
- **Commission Restructuring:** Reallocate Q3 hiring budgets toward the Online Sales department, which demonstrated the highest Revenue-to-Salary ROI ratio in the performance matrix.
- **Future ML Integration:** Expand the current infrastructure to support Python-based Machine Learning (e.g., Scikit-Learn) for Predictive Sales Forecasting and personalized Product Recommendation Engines based on historical cart associations.